

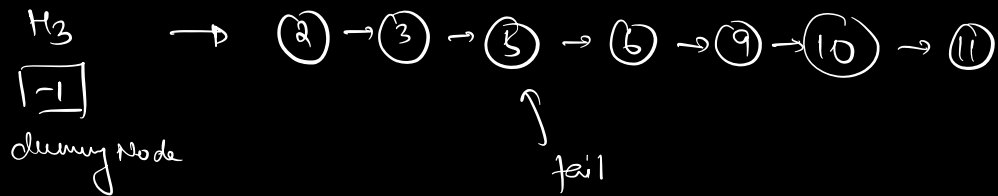
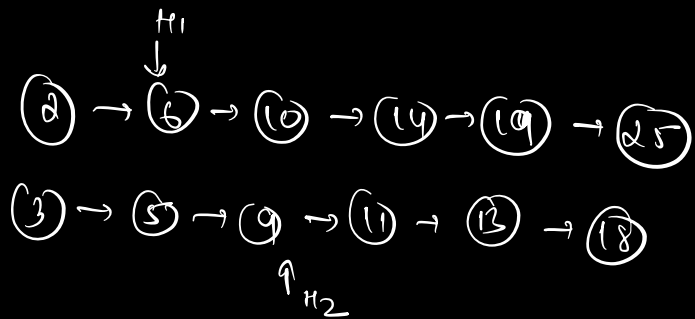
Q1. Merge two sorted LLs, and return head of resulting sorted LL.

H1 $\rightarrow$ (2) $\rightarrow$ (6) $\rightarrow$ (10) $\rightarrow$ (14) $\rightarrow$ (19) $\rightarrow$ (25)

H2 $\rightarrow$ (3) $\rightarrow$ (5) $\rightarrow$ (9) $\rightarrow$ (11) $\rightarrow$ (13) $\rightarrow$ (18)

result $\Rightarrow$ H3 $\rightarrow$ (2) $\rightarrow$ (3) $\rightarrow$ (5) $\rightarrow$ (6) $\rightarrow$ (9) $\rightarrow$ (10) $\rightarrow$ (11) $\rightarrow$ (13) $\rightarrow$ (14) $\rightarrow$ —

Soln



H1

tail.next = h1

h1 = h1.next

tail = tail.next

H2

tail.next = h2

h2 = h2.next

tail = tail.next

Node merge (h1, h2) {

Node h3 = null, tail = null

h3 = new Node(-1), tail = h3

if (h1 == null) return h2

if (h2 == null) return h1

if (h1.data <= h2.data) {

h3 = h1

h1 = h1.next

tail = h3

else

h3 = h2

h2 = h2.next

tail = h3

}

while (h1 != null || h2 != null) {

if (h1.data <= h2.data) {

{ h1's logic }

else

h2's logic

}

return h3 | h3.next

↓

no dummy node

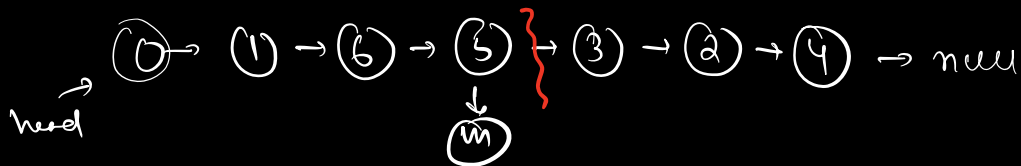
↓

been dummy node

don't want
take a
dummy
node

$$\boxed{\begin{array}{l} \text{TC} \Rightarrow O(N+N) \quad \text{SC} \Rightarrow O(1) \end{array}}$$

* Sort a given LL (merge sort)



Node mergeSort(head) {

if (head == null || head.next == null) return head;

Node m = middle(head);

Node h2 = m.next;

m.next = null;

head = mergeSort(head);

h2 = mergeSort(h2);

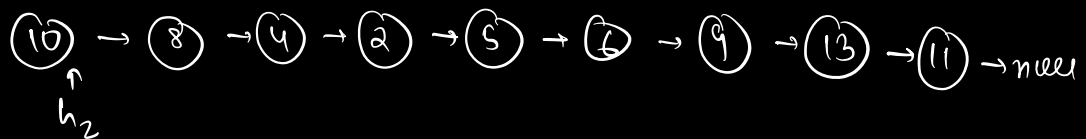
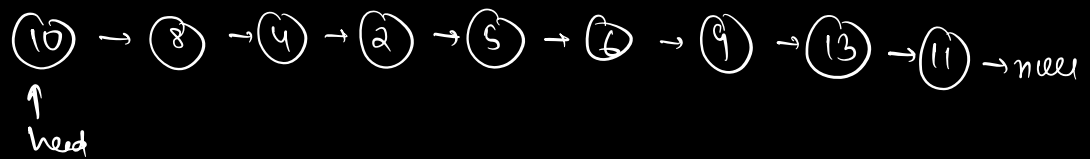
return merge(head, h2);

}

TC $\Rightarrow O(N \log N)$

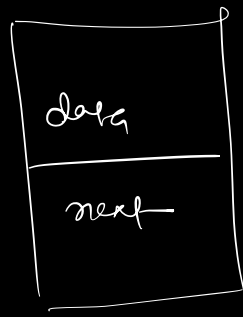
SC $\Rightarrow O(1)$ + recursion stack $O(\log N)$

Q3. Clone a LL:-



: Doubly LL:

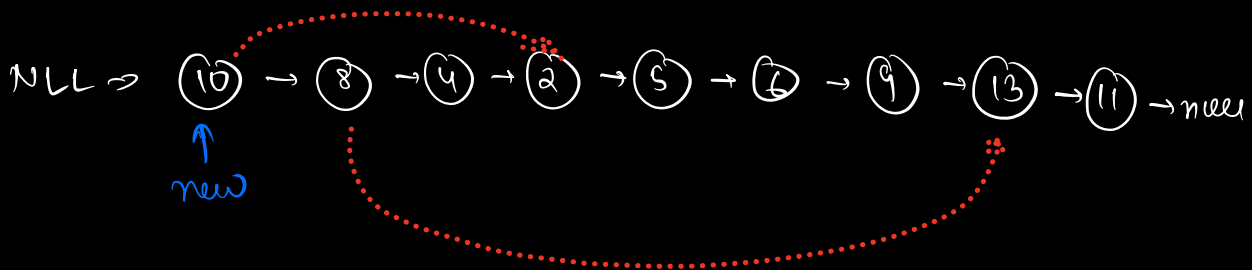
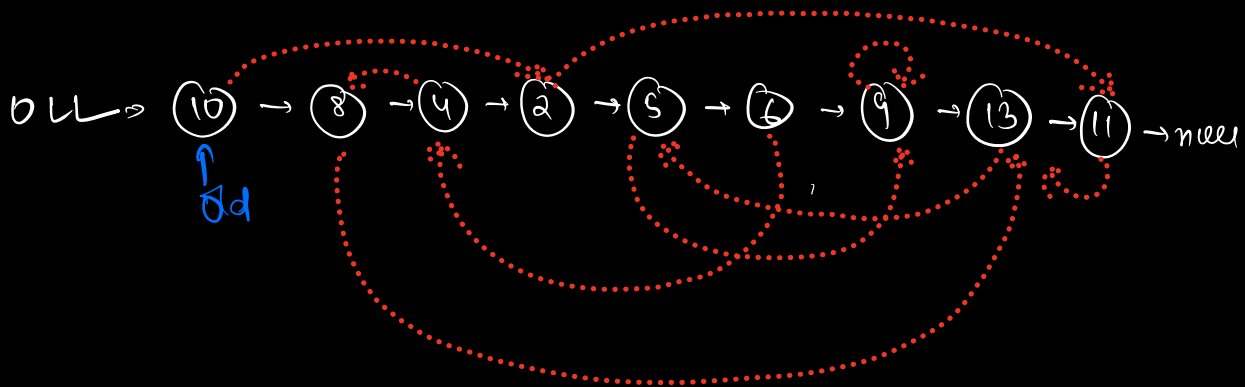
Singly LL $\rightarrow$



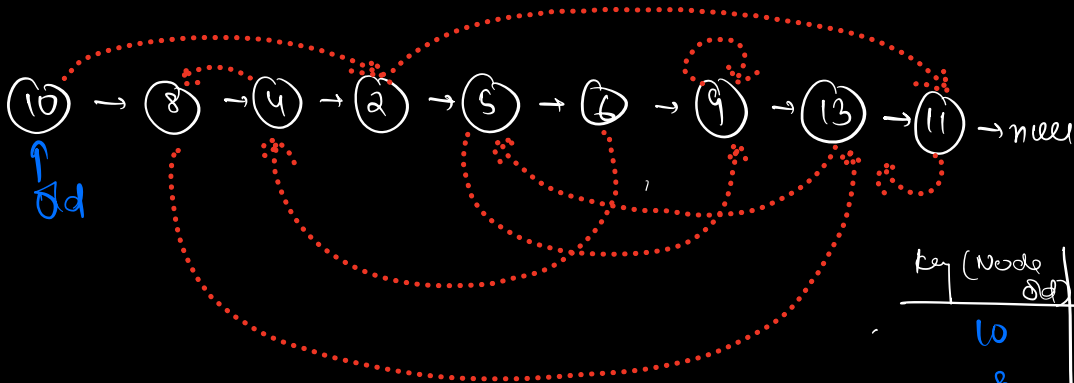
Doubly LL $\rightarrow$



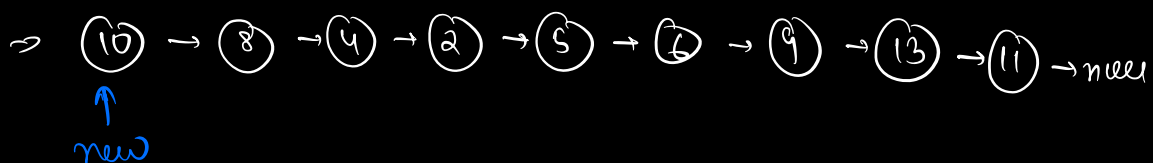
Q4. Deep clone a LL with next and random ptr.



$\Rightarrow \underline{O(N^2)}$



key (Node id)	value Node (new)
10	10
8	8
4	4
2	2



$H_2 = \text{new Node}(1);$

$\text{tail} = H_2$

$\text{Node } \text{old} = \text{head};$

$\text{while}(\text{old} \neq \text{null}) \{$

$\text{temp} = \text{new Node}(\text{old.data});$

$\text{tail.next} = \text{temp}; \text{tail} = \text{tail.next}$

$\text{HM.insert}(\text{old}, \text{temp})$

$\text{old} = \text{old.next};$

$\}$

$H_2 = H_2.next;$

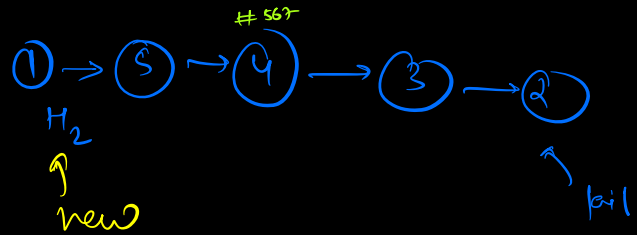
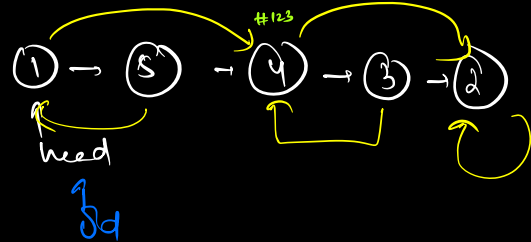
$\text{old} = \text{head};$

$\text{new} = H_2;$

$\text{HM} \langle \text{Node}, \text{Node} \rangle$

$\downarrow$
 old
 LL

$\downarrow$
 new
 LL



key(old)	value(new)
(#123) 1	1 (#567)
5	5
4	4
3	3
2	2

→ random ptr

while (old != null) {

 x = HM.get(old.random)

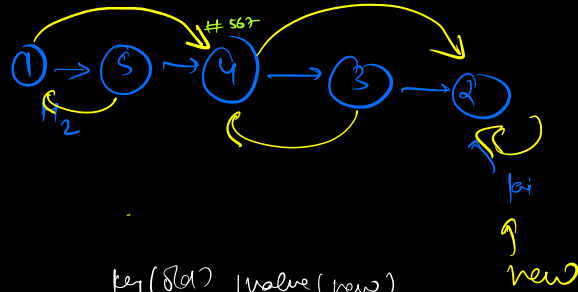
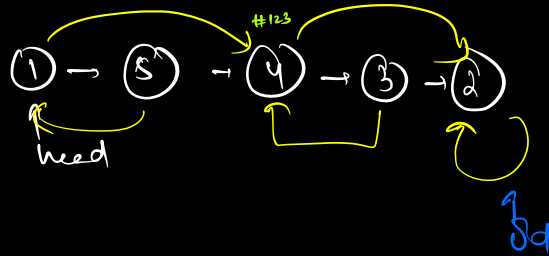
 new.random = x

 old = old.next

 new = new.next

}

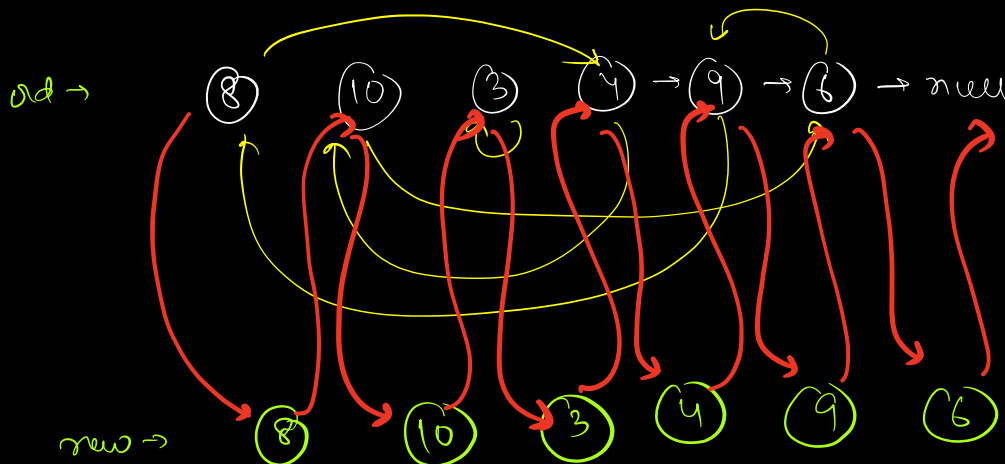
new.next = null



key (old)	value (new)
1 (#123)	1 (#567)
5	5
4	4
3	3
2	2

$T \Rightarrow O(N)$

$SC \Rightarrow O(N)$

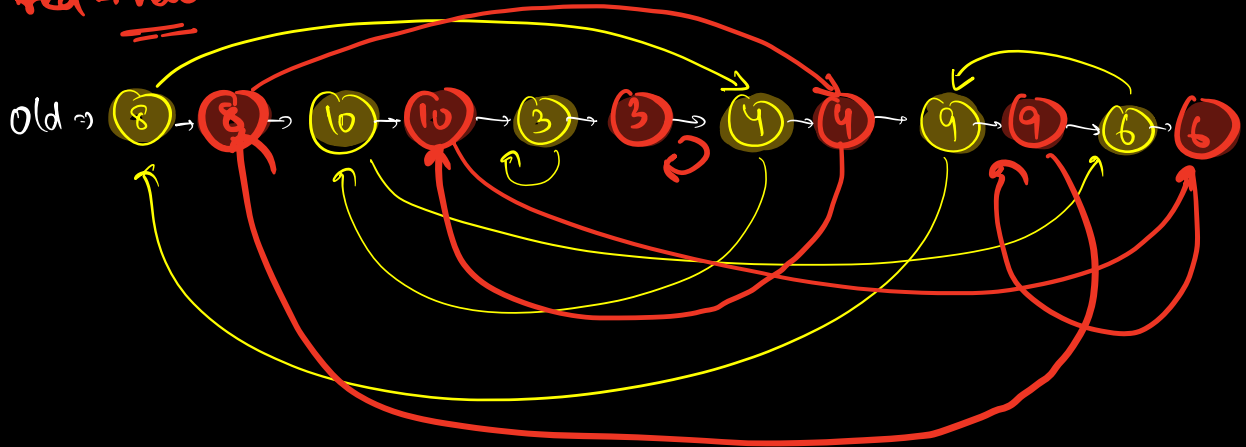


yellow → old

old 8.next → 10

old.next.next → 10

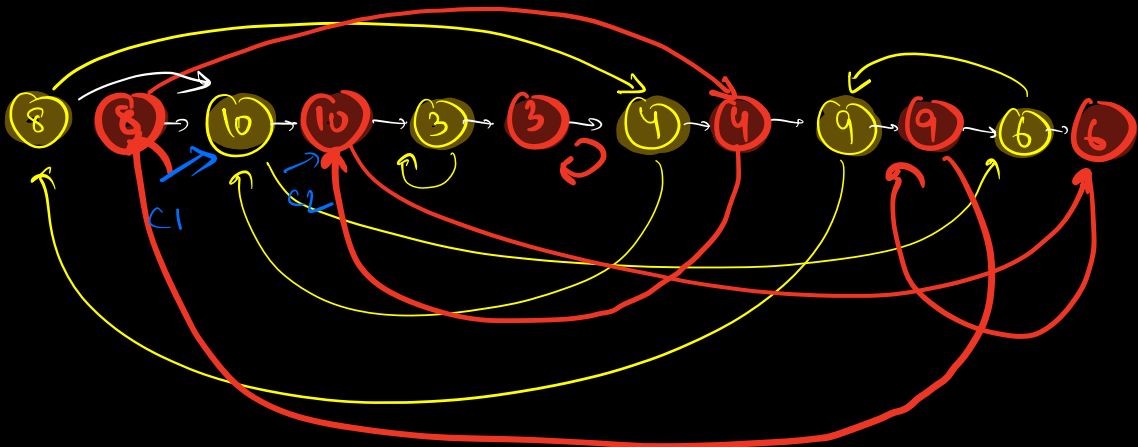
red $\rightarrow$ new



```
new.random = Id.random.next;
```

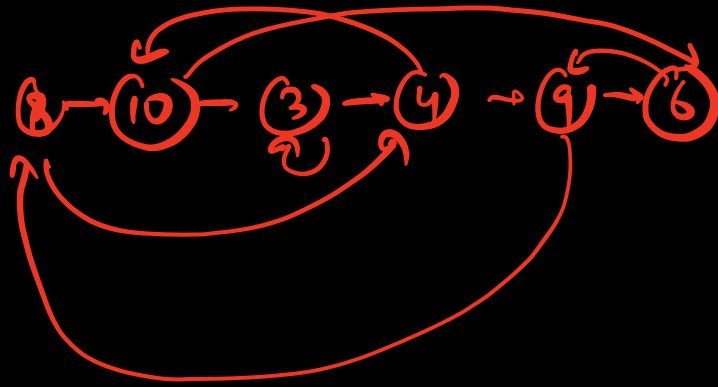
$$\text{odd} = \text{odd} \cdot \text{next} \cdot \text{next}$$

new = new * next * next!

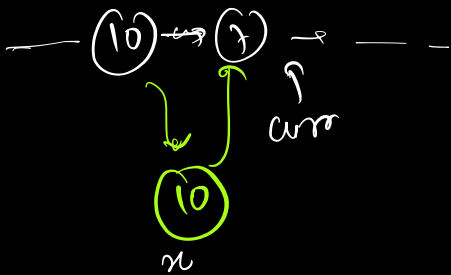

$$C_i.next = C_i.next.next;$$

```
C2.next = C2.next.next;
```





Step 1 → Create new nodes and insert in old LL



curr = head;

while (curr != null) {

x = new Node (curr.data);

x.next = curr.next;

curr.next = x

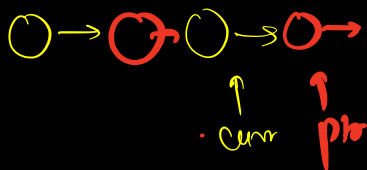
curr = curr.next.next

}

Step 2 → update the random ptrs. of new nodes :-

curr = head;

ptr = head.next;



while (curr != null) {

ptr.random = curr.random.next

curr = curr.next.next

ptr = ptr.next.next

}

Steps $\rightarrow$ delinking the new nodes to form a LL

curr $\rightarrow$ head;

Node ans = curr->next;

ptr = curr->next;

while (curr != null) {

curr->next = curr->next->next

curr->next;

ptr->next = ptr->next->next

ptr->next;

}

return ans;

TC $\Rightarrow O(N)$ SC $\Rightarrow O(1) \rightarrow$ except the new LL

LRU Cache



least recently used cache

Cache :

temporary / small / fast → storage

capacity
5

7 3 9 2 6 10 14 2 10 15 8 14
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
X X X X X X ✓ ✓ X X ✓

(✓) → HIT → get data from cache present in cache
(X) → MISS → not present in cache

cache ⇒

7 3 9 2 6

↓ (-7) + (10)

3 9 2 6 10

↓ (-3) + (14)

9 2 6 10 14



9 6 10 14 2



9 6 14 2 10

↓

6 14 2 10 15

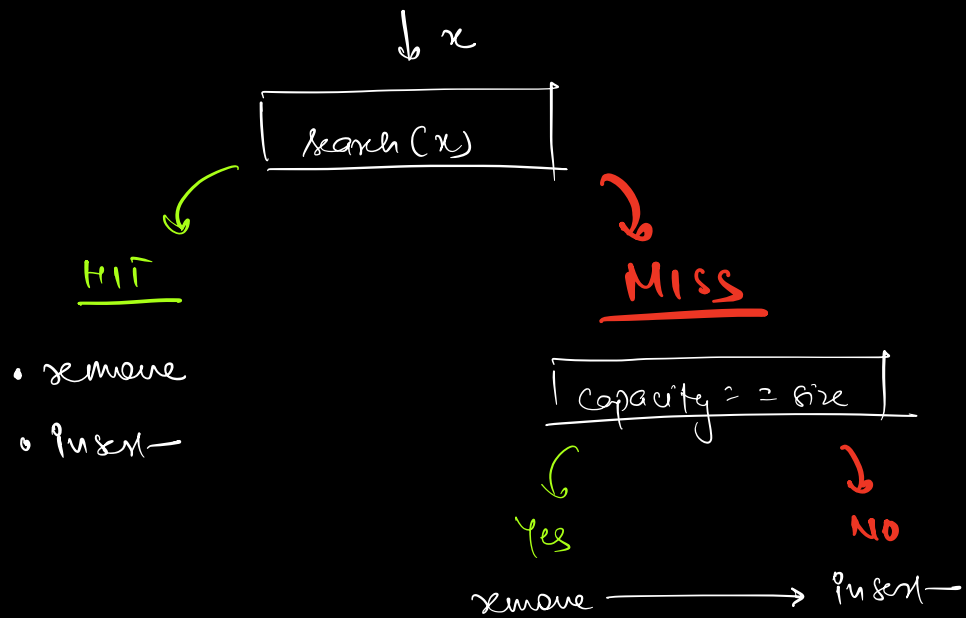
↓

14 2 10 15 8

↓

2 10 15 8 14

- search
- remove
- insert



	Arrays	LL	Hashmap + LL	DLL + Hashmap
Search	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Insert	$O(1)$	$O(1)$	$O(1)$	$O(1)$
remove	$O(N)$	$O(1)$ * scanning already reached to that node	$O(1)$ prev ptr.	$O(1)$

DLL →

```

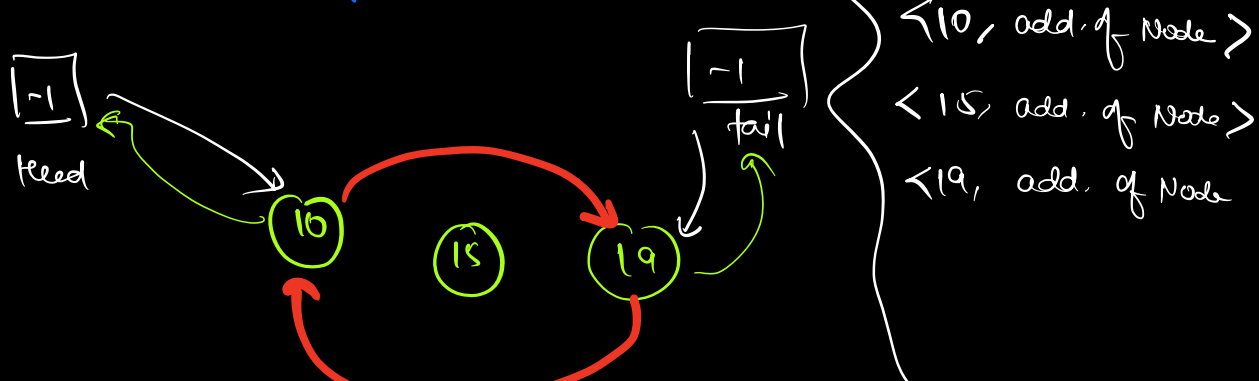
class Node {
    int data;
    Node next;
    Node prev;
}

```

10 15 19 15 20 18 23 20 19 17 17

↑ ↑ ↑ ↑

× × × ✓



addTail(x) {

$x.\text{next} = \text{tail}$

$x.\text{prev} = \text{tail}.\text{prev}$

$\text{tail}.\text{prev} = x$

$x.\text{prev}.\text{next} = x$

remove(x) {

$x.\text{prev}.\text{next} = x.\text{next};$

$x.\text{next}.\text{prev} = x.\text{prev};$

}